

FB15k-237 Link Prediction: Beyond Default KGE—Hard Negatives, GNNs, and Relation-Level Analysis

Thalia Delhaise student no. 66937

Lenny Briclet student no. 66926

André Fonseca student no. 58225

Rafael Gufler student no. 66081

Deep Learning & Knowledge Graphs — unified project (all four members).

Problem and motivation. A *knowledge graph* collects facts as triples (h, r, t) . *Link prediction* scores queries $(h, r, ?)$ and $(?, r, t)$ using a learned $f_\theta(h, r, t) \in \mathbb{R}$ [2]. **Research question:** under **matched** training conditions on FB15k-237, where do **RotatE vs. TransE** differ in **filtered** ranking—only in aggregate, or mainly on **specific relation / degree regimes**? Second, does **custom hard-negative training** (our PyTorch code) change those gaps vs. library default corruption? Third, how does a **relational GNN** link predictor compare to embedding-only KGE on the same splits? **What we aim to learn:** not a new SOTA claim, but **quantified** trade-offs (accuracy vs. compute; frequent vs. rare relations), and when each modeling choice helps or hurts.

Pipeline (build the KG, then predict). The project matches both courses in order: **(1) Dataset $\rightarrow$ graph.** We use the public **FB15k-237** benchmark [3] - released as **real triple files** (train/validation/test splits). That release is our dataset: thousands of facts about real-world entities. **Building** the KG means ingesting those triples, mapping strings to entity/relation IDs, forming the edge set $\mathcal{T}$ and the split, and loading the result into PyKEEN [1]; we may also export a small RDF or edge-list slice for figures or the KG report. **(2) Prediction.** We then train models only on $\mathcal{T}_{\text{train}}$ and evaluate on held-out test triples (next paragraph). We do not invent a private ontology from scratch, but we still **materialize** the full multi-relational graph the methods run on.

What the graph looks like. Nodes are entities; each fact is a **directed, labeled** link $h \xrightarrow{r} t$. Many relation types exist, and the same pair (h, t) can appear under different r . Equivalently, $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ with $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$. We keep the official benchmark partition unchanged (no extra subsampling).

Task and evaluation protocol. Training uses only $\mathcal{T}_{\text{train}}$: we minimize a **margin/ranking loss** and draw **negative** triples by corrupting head or tail (standard pipeline [2, 3]). At test time, each $(h, r, t) \in \mathcal{T}_{\text{test}}$ was unseen during training; we score all candidate completions and record the rank of the true entity. **Filtered** metrics [2]: when ranking the true tail for $(h, r, ?)$, any other tail t' with $(h, r, t') \in \mathcal{T}_{\text{train}} \cup \mathcal{T}_{\text{val}} \cup \mathcal{T}_{\text{test}}$ is treated as ambiguous (not penalized as an error); the same for head prediction. We average results over head and tail prediction.

Scale and Milestone 1 characterization. FB15k-237 counts: **14,541** entities, **237** relations, **272,115 / 17,535 / 20,466** train/val/test triples. By Milestone 1 we add **dataset statistics** beyond counts: relation-frequency and (head/tail) **degree distributions**, plus short examples of **one-to-one / one-to-many / many-to-one** style patterns to motivate later error analysis. Optional second graph (e.g. OGB); FB15k-237 stays primary.

Method (planned). Core comparison (matched where possible): TRANSE [2] vs. ROTATE [3] with the **same** splits, training budget, embedding size d , and filtered evaluator; PyKEEN [1] for reproducible **data loading** and **default** KGE runs. **Controlled ablation axis:** negative-sampling / corruption strategy (default vs. our extension in (1)); if time, a second axis (e.g. d or training epochs) for efficiency–accuracy curves.

Metrics, relation-wise analysis, and error study. Aggregate: filtered MRR and Hits@1, Hits@3, Hits@10 (head, tail, then averaged). **Relation-wise:** same metrics **sliced** by relation frequency or degree bins (addresses “only a single number” risk). **Qualitative error analysis:** inspect successes/failures and relate them to relation ambiguity, tail/head frequency, or pattern type. **Conceptual note:** we keep **filtered** ranking as primary and briefly contrast why raw ranks would be optimistic [2]. Training/validation curves for all models.

Our contribution vs. library defaults. PyKEEN gives a fair **baseline pipeline**; our **own implementation** (PyTorch / PyG or DGL) covers the **custom hard-negative** module, optional **bootstrap** code for CIs, **slice** evaluation scripts, and the **GNN** trainer—so the project is not “only two API calls.”

Additional experiments (DL depth). **(1) Custom negative sampling.** Learned scorer g_ϕ and/or periodic hard-negative mining from current embeddings; **ablate** against uniform/Bernoulli corruption. **(2) Slice + uncertainty.** Relation/degree slices; optional bootstrap CIs on the test query set. **(3) GNN track.** Shallow R-GCN or COMPGCN link predictor on the **same** splits (independent of (1)). **Final readout:** default KGE vs. KGE+custom negatives vs. GNN, plus relation-wise tables and a short error discussion.

References

- [1] M. Ali, M. Berrendorf, C. T. Hoyt, L. Vermue, S. Sharifzadeh, V. Tresp, and J. Lehmann. PyKEEN 1.0: A python library for training and evaluating knowledge graph embeddings. *Journal of Machine Learning Research*, 22(82):1–6, 2021. [JMLR page](#) ([PDF](#)); [documentation](#).
- [2] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, 2013. [NeurIPS paper page](#); [PDF](#).
- [3] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*, 2019. [OpenReview \(paper + PDF\)](#).